

Ambiente de Desenvolvimento Integrado AURORA: Compilação e Simulação para o Processador Soft-Core SAPHO

Chrysthofer A. A. Afonso* Gabriel A. Torres*
Pedro H. S. R. Santos* Luciano M. de Andrade Filho*

* Departamento de Engenharia Elétrica, Universidade Federal de Juiz de
Fora, MG, (e-mail: chrysthofer.afonso@estudante.ufjf.br)

Abstract: FPGAs are widely employed in embedded systems where soft-core processors execute complex algorithms. The SAPHO dynamic processor reduces logic-area overhead by synthesizing only the hardware resources each application requires. SAPHO's Integrated Development Environment (IDE) predecessor (named IDE V1) offered limited automation and no simulation support. This paper presents AURORA, a modular and responsive IDE built on Electron that consolidates the SAPHO design flow. The platform automates the workflow from C $\pm$ source-code editing to RTL netlist visualization via PRISM and waveform analysis via GTKWave, within a unified interface. Available as an open-source project, AURORA supports graduate research and undergraduate courses at the Federal University of Juiz de Fora (UFJF).

Resumo: As FPGAs são amplamente utilizadas em sistemas embarcados nos quais processadores *soft-core* executam algoritmos complexos. O processador dinâmico SAPHO reduz o desperdício de área lógica ao sintetizar apenas os recursos de *hardware* requeridos por cada aplicação. Sua interface predecessora, a IDE V1, apresentava automação limitada e não integrava ferramentas de simulação. Este trabalho apresenta a AURORA, um ambiente de desenvolvimento integrado (IDE) modular e responsivo, baseado em *Electron*, que consolida o fluxo de projeto do SAPHO. A plataforma automatiza o desenvolvimento desde a edição do código-fonte em C $\pm$ até a visualização do *netlist* RTL no PRISM e a análise de formas de onda no GTKWave, em um ambiente unificado. Disponível como código aberto, a AURORA apoia pesquisas de pós-graduação e atividades curriculares na Universidade Federal de Juiz de Fora (UFJF).

Keywords: FPGA; soft-core processor; hardware design automation; compiler toolchain; AURORA IDE.

Palavras-chave: FPGA; processador *soft-core*; automação de projeto de *hardware*; cadeia de compiladores; AURORA IDE.

1. INTRODUÇÃO

As FPGAs (*Field-Programmable Gate Arrays*) consolidaram-se como plataformas relevantes para o processamento de cálculos complexos, destacando-se pelo paralelismo massivo e pela eficiência energética (Compton and Hauck, 2002). Em cenários de alta exigência computacional, como no experimento ATLAS conduzido no CERN (Organização Europeia para a Pesquisa Nuclear, em francês *Conseil Européen pour la Recherche Nucléaire*), esses dispositivos constituem o núcleo dos sistemas de gatilho (*trigger*) e de aquisição de dados (DAQ). Eles são responsáveis por filtrar, em tempo real, o fluxo de informações geradas pelas colisões de partículas, selecionando e registrando apenas os eventos de interesse físico (ATLAS Collaboration, 2008). Essa adoção se deve ao fato de as FPGAs permitirem o mapeamento direto de algoritmos em circuitos de *hardware* customizáveis, maximizando o desempenho das operações.

Quando o projeto demanda algoritmos extensos ou cálculos em ponto flutuante, é prática comum recorrer a

processadores embarcados (*soft-cores*) programados via *software*. A principal limitação dessa abordagem é a perda de velocidade e o desperdício de área lógica da FPGA, uma vez que a arquitetura padrão do processador é instanciada integralmente, independentemente de sua utilização efetiva. Para contornar essa limitação, desenvolveu-se o SAPHO (*Scalable Architecture Processor for Hardware Optimization*): um processador cuja arquitetura é gerada sob demanda (Kapisch et al., 2026). Programado por meio de uma linguagem de médio nível denominada C $\pm$, o SAPHO sintetiza exclusivamente os recursos de *hardware* requeridos pelo código *Assembly* final da aplicação, assegurando a otimização física do dispositivo.

Para viabilizar a parametrização e o desenvolvimento de projetos baseados no SAPHO, concebeu-se inicialmente uma Interface de Desenvolvimento Integrada (IDE), referenciada como Versão 1 (V1) e implementada em C# (*C-Sharp*). Essa primeira ferramenta permitia configurar características estruturais do processador – como o uso de aritmética em ponto fixo ou flutuante e a largura da palavra

– e instanciar na FPGA apenas os circuitos necessários (Kapisch et al., 2026). A adoção contínua da ferramenta por distintos grupos de trabalho evidenciou, com o tempo, a necessidade de um ambiente mais abrangente. Nesse contexto, desenvolveu-se a IDE AURORA (*Advanced Utility Running Optimized Resource Architectures*), baseada na tecnologia *Electron*. Essa nova plataforma expande o fluxo de projeto, permitindo testar toda a cadeia de desenvolvimento em um único ambiente: desde a escrita do código-fonte e a simulação interna, até a geração do código Verilog *top-level* e a pré-visualização do circuito RTL (*Register-Transfer Level*) em formato esquemático. O uso de *softwares* proprietários de fabricantes de FPGAs fica, assim, restrito à etapa final de síntese e roteamento físico.

Todo este ecossistema é fruto de um esforço de desenvolvimento de código aberto conduzido pelo Núcleo de Instrumentação e Processamento de Sinais da Universidade Federal de Juiz de Fora (NIPS-CERN), em colaboração com o CERN. A AURORA encontra-se integrada à rotina de projetos do laboratório, sendo empregada no desenvolvimento de métodos de processamento *on-line* de sinais para instrumentação nuclear e qualidade de energia elétrica. Além de fundamentar diversas pesquisas de pós-graduação, as ferramentas SAPHO e AURORA estão em processo de integração ao currículo de graduação da UFJF, na disciplina de Dispositivos Lógicos Programáveis (DLP) da Faculdade de Engenharia Elétrica com habilitação em Sistemas Eletrônicos. Todos os compiladores e códigos-fonte estão disponíveis no portal do laboratório¹.

Este artigo está organizado da seguinte forma. A Seção 2 apresenta a visão arquitetural do processador SAPHO, evidenciando sua modularidade e a alocação dinâmica de unidades funcionais. A Seção 3 fundamenta a adoção de tecnologias *web* e do *framework Electron* como base arquitetural para ferramentas de *hardware*. A Seção 4 descreve a IDE AURORA, detalhando suas funcionalidades operacionais e os recursos de integração do fluxo de projeto. A Seção 5 estabelece um comparativo entre a AURORA e a IDE V1, destacando os avanços obtidos. Por fim, a Seção 6 sumariza as contribuições do trabalho.

2. O PROCESSADOR SAPHO

O SAPHO é um processador do tipo *soft-core* desenvolvido pelo laboratório NIPS-CERN. Ele foi projetado para otimizar o uso de recursos em FPGAs, permitindo que a arquitetura seja moldada conforme as necessidades de cada aplicação. Diferentemente de processadores comerciais de arquitetura fixa, o SAPHO utiliza uma abordagem dinâmica em que os recursos são alocados automaticamente em tempo de compilação. Dessa forma, apenas os elementos lógicos necessários para executar o algoritmo do usuário são instanciados no *hardware*, reduzindo o custo computacional do projeto (Kapisch et al., 2026). A aplicabilidade e o potencial deste modelo foram validados em cenários de alta carga de trabalho, destacando-se o uso do SAPHO como núcleo de processamento para a geração de fractais em FPGA Andrade Filho et al. (2025). Este caso de uso é representativo pois a complexidade intrínseca das formas fractais decorre de uma recursividade matemática que exige do *hardware* o suporte a fluxos de dados realimentados

e alta precisão em operações sucessivas. Ao traduzir essa característica algorítmica para a arquitetura, a renderização atesta a capacidade do processador em sustentar o desempenho em laços iterativos densos, validando a estabilidade da alocação de suas unidades funcionais diante de algoritmos que demandam processamento ininterrupto e convergência numérica.

2.1 Arquitetura do Processador

A arquitetura do SAPHO foi concebida para aliar alta taxa de processamento a uma utilização reduzida de recursos físicos na FPGA. Essa estrutura modular e escalonável viabiliza a integração de diversas unidades funcionais especializadas sem a necessidade de redesenhar o núcleo de processamento. As principais unidades responsáveis pela execução das instruções e pela manipulação do fluxo de dados são:

- **Unidade de Controle:** Responsável por gerenciar o fluxo de instruções e coordenar as operações entre os módulos do processador, assegurando que os sinais de controle sejam gerados nos ciclos de *clock* adequados.
- **Unidade Lógica e Aritmética (ULA):** Encarregada do processamento matemático e lógico, a ULA do SAPHO é parametrizada automaticamente com base nas instruções oriundas do compilador *Assembly* e nas definições de projeto estabelecidas pelo usuário – como o número de portas de I/O (entradas e saídas) e a largura da palavra de dados. A ULA foi estruturada para executar operações em um único ciclo de clock, otimizando o fluxo da estrutura em *pipeline* do processador.
- **Unidade de Memória e Endereçamento Relativo:** Encapsula os mecanismos de acesso aos dados. O endereçamento relativo é instanciado dinamicamente quando matrizes ou vetores são utilizados no programa, indexando os elementos por meio de um *offset* a partir da posição inicial de memória. Isso permite o acesso direto ao dado desejado, eliminando lógicas adicionais implementadas via *software*.

Essa organização interna permite que o SAPHO se adapte aos requisitos de cada projeto e maximize o aproveitamento da área útil da FPGA, convertendo algoritmos extensos em uma infraestrutura de *hardware* compacta. Dessa forma, o SAPHO pode ser visto como um sistema híbrido que combina *High Level Synthesis* (HLS) – onde o hardware é definido estaticamente por uma linguagem de programação de software – com fluxo de processamento que segue a dinâmica da arquitetura de processador embarcado.

2.2 Dinâmica Operacional e Pipeline

A dinâmica de funcionamento do SAPHO fundamenta-se na sinergia entre sua infraestrutura física e sua lógica de controle. Para otimizar o fluxo de processamento e reduzir a latência, a arquitetura foi estruturada com um *pipeline* de três estágios síncronos, conforme ilustrado na Figura 1: busca (*Fetch* – F), decodificação (*Decode* – D) e execução (*Execute* – E) (Kapisch et al., 2026).

O diferencial dessa arquitetura é a capacidade de executar qualquer algoritmo sem interromper a cadeia sequencial do *pipeline* ($F \rightarrow D \rightarrow E$), mesmo na presença de instruções

¹ <https://www.nipscern.com/>

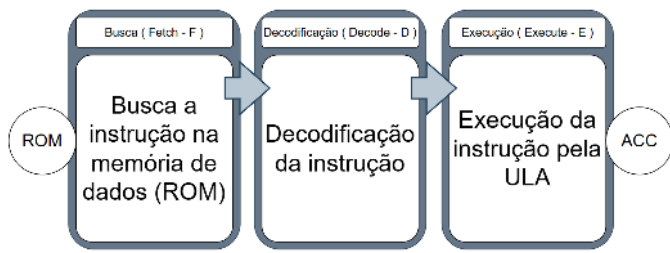


Figura 1. Diagrama dos estágios de *Pipeline* executados pelo SAPHO.

de salto condicionais ou chamadas de função (Kapisch et al., 2026). A continuidade desse fluxo é viabilizada pelo projeto do processador, no qual a ULA atua como um bloco combinacional, acumulando uma operação lógico-aritmética no registrador acumulador (ACC) a cada ciclo de *clock*. Como as instruções necessitam buscar apenas um operando na memória – sendo o segundo operando o próprio ACC, o processador otimiza o fluxo de dados sem introduzir quebras no *pipeline*.

2.3 Infraestrutura de Software

Para viabilizar a programação e a geração customizada do *hardware*, o SAPHO conta com uma infraestrutura de *software* dedicada. O sistema opera por meio de uma IDE que executa os compiladores C $\pm$ e *Assembler*. O fluxo de conversão do código em circuito ocorre em duas etapas. Na primeira, o código em C $\pm$ é processado por um compilador construído com as ferramentas Flex e Bison da GNU (Levine, 2009), que interpreta a lógica do algoritmo e gera o respectivo código em linguagem *Assembly*. Na segunda etapa, o compilador *Assembler* converte esse conjunto de instruções em arquivos de descrição de *hardware* em Verilog. Além da tradução lógica, essa camada gerencia os parâmetros operacionais da ULA, dimensiona as memórias de programa e de dados, e instancia as portas de entrada e saída requeridas pelo algoritmo. Historicamente, esse ecossistema operava por meio da IDE V1 cujas limitações motivaram a adoção de uma nova arquitetura de software para o desenvolvimento de uma interface modernizada, conforme discutido a seguir.

3. ARQUITETURA DE SOFTWARE BASEADA EM TECNOLOGIAS WEB

Historicamente, o desenvolvimento de interfaces gráficas para ferramentas de Electronic Design Automation (EDA) e Ambientes de Desenvolvimento Integrado (IDEs) era condicionado ao uso de linguagens compiladas e APIs proprietárias de cada sistema operacional. Softwares desse nicho frequentemente utilizavam frameworks como .NET/C++ (Windows) ou Qt/C++, que, apesar de oferecerem alto desempenho, criavam um "aprisionamento tecnológico" (vendor lock-in), dificultando a portabilidade e exigindo manutenções distintas para cada plataforma. Um exemplo prático desse cenário foi a IDE V1 do SAPHO, cuja dependência do ecossistema .NET limitava sua evolução visual e seu alcance multiplataforma.

Para contornar essas limitações de interoperabilidade e flexibilidade arquitetural, o desenvolvimento de *software*

moderno tem adotado a transição para aplicações *desktop* multiplataforma baseadas em tecnologias *web*. Essa mudança de paradigma fundamenta-se na sinergia de três pilares centrais: o HTML (*HyperText Markup Language*), que define a estruturação semântica e a hierarquia dos painéis visuais; o CSS (*Cascading Style Sheets*), responsável pela estilização geométrica e cromática, permitindo a criação de interfaces responsivas; e o JavaScript, que atua como o motor lógico para o controle de eventos e a manipulação dinâmica da interface em tempo de execução.

Contudo, navegadores *web* tradicionais executam o código JavaScript em um ambiente isolado (*sandbox*) por razões de segurança, impedindo o acesso direto aos recursos do sistema operacional, como o sistema de arquivos e o escalonamento de processos. Como as cadeias de compiladores (*toolchains*) de processadores exigem a leitura, escrita e execução contínua de binários locais, uma aplicação baseada puramente em navegadores seria inviável para o fluxo de projeto em FPGAs. É neste hiato tecnológico que se insere o *framework Electron* (Yang et al., 2025).

O *Electron* atua como uma ponte arquitetural, unificando dois ambientes de execução distintos em um único executável, conforme ilustrado no diagrama de blocos da Figura 2. Essa conexão é viabilizada pela Comunicação Interprocessos (IPC - Inter-Process Communication), um mecanismo de troca de mensagens que permite ao processo de renderização solicitar ações de baixo nível do processo principal. Em sua camada de *frontend* (processo de renderização), ele incorpora o motor estrutural Chromium, responsável por interpretar as tecnologias *web* (HTML, CSS e JavaScript) com alto desempenho gráfico. Paralelamente, em sua camada de *backend* (processo principal), ele embute o ambiente de execução Node.js.

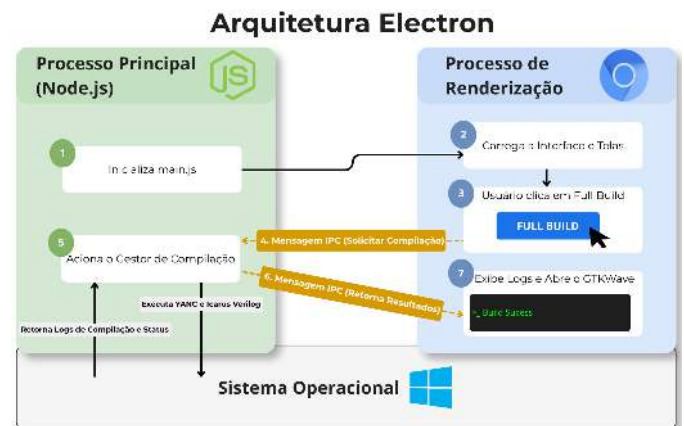


Figura 2. Arquitetura do *Electron* baseada nos ambientes Chromium e Node.js.

A modernização desse paradigma ocorre com a adoção de tecnologias que unem a versatilidade do desenvolvimento *web* à robustez do Node.js. Essa integração permite que a aplicação usufrua da riqueza estética e da vasta biblioteca de componentes do ecossistema front-end, enquanto o Node.js provê acesso de baixo nível ao sistema operacional. O Node.js atua como o núcleo operacional da ferramenta, gerenciando integralmente o ciclo de vida dos arquivos do SAPHO: desde descritores de hardware, passando pelos arquivos de configuração e orquestrando a execução de

processos filhos. Com isso, a invocação de simuladores e compiladores ocorre de maneira intrínseca ao fluxo de trabalho, eliminando as barreiras de execução de ferramentas externas. Essa base tecnológica foi o fator determinante para a construção da AURORA, a nova IDE para o processador SAPHO, apresentada na seção seguinte.

4. A IDE AURORA

A AURORA atua como um ambiente de desenvolvimento, interface e controle integrado, distinguindo-se das ferramentas EDA (*Electronic Design Automation*) tradicionais, que têm foco na síntese e no roteamento físico. Ela é a principal interface gráfica do ecossistema de processamento desenvolvido pelo NIPS-CERN, atuando como o *frontend* que orquestra a cadeia de compiladores YANC (*Yet Another Compiler*). Conceitualmente, o sistema completo é definido pela composição em que o processador SAPHO resulta da integração entre a AURORA e o YANC. A Figura 3 ilustra a tela inicial da IDE, onde o usuário seleciona o modo de trabalho desejado.



Figura 3. Tela inicial da IDE AURORA com os modos de trabalho disponíveis.

Desenvolvida com o *framework Electron* e tecnologias *web*, a AURORA oferece um ambiente de parametrização e programação estruturada. Seu fluxo de trabalho é dividido em duas frentes configuráveis: o Modo Processador e o Modo Projeto.

4.1 Modo Processador e Linguagens Suportadas

No Modo Processador, o usuário inicia a modelagem de seu *hardware* definindo os parâmetros fundamentais da unidade de processamento. A interface permite configurar, de forma visual, o número de *bits* da arquitetura, o tamanho da mantissa, do expoente, além da quantidade de portas de entrada e saída.

A programação do comportamento do processador conta com três linguagens distintas, suportadas pelo editor de código integrado da AURORA:

- **C $\pm$ (CMM):** Linguagem de médio nível proprietária desenvolvida pelo NIPS-CERN. Trata-se de um *subset* da linguagem C com funcionalidades voltadas para a física de altas energias, incluindo suporte nativo à notação matemática *bra-ket*, utilizada para representar

estados quânticos e calcular probabilidades (Dirac, 1958).

- **Assembly:** Disponível para programadores que necessitam de otimizações de baixo nível e controle sobre o ciclo de máquina.
- **Verilog:** Permite a descrição direta em nível de transferência de registradores (RTL), oferecendo controle completo ao projetista.

4.2 Modo Projeto e Integração de Pipeline

Após a criação e programação de múltiplos processadores individuais, o desenvolvedor acessa o Modo Projeto (*Project Mode*). Nesse ambiente, os processadores previamente desenvolvidos podem ser instanciados e interconectados, formando um *pipeline* completo de processamento de dados. É também nessa etapa que o *testbench* do sistema é definido, preparando o circuito para a validação conjunta.

4.3 Toolchain YANC e Fluxo de Compilação Automatizado

O núcleo de tradução do ecossistema é o YANC, um conjunto de compiladores desenvolvidos em C, com o auxílio da *GNU Compiler Collection* (GCC) e das ferramentas GNU Bison (gerador de analisador sintático) e Flex (analisador léxico) (Donnelly and Stallman, 2021; Levine, 2009). A Figura 4 apresenta o editor de código integrado da AURORA, que oferece suporte à sintaxe de todas as linguagens disponíveis, com coloração e autocompletar por meio do *Monaco Editor* – o mesmo núcleo utilizado no *Visual Studio Code*.

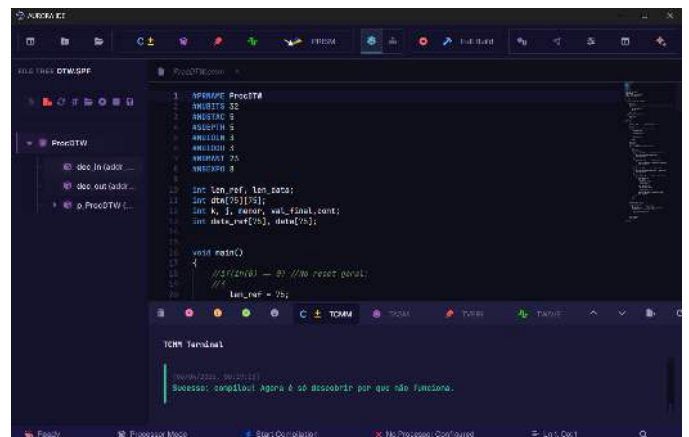


Figura 4. Editor de código integrado da AURORA com exemplo de programa escrito em C $\pm$.

A AURORA abstrai a complexidade dessa cadeia por meio de uma interface em que o usuário define os parâmetros de compilação – modo paralelo ou seriado, frequência de *clock*, quantidade de ciclos de simulação – e aciona a rotina de forma automatizada. O processo ocorre nas seguintes etapas sequenciais:

- (1) **Tradução para Assembly:** O código-fonte em C $\pm$ (.cmm) é processado pelo executável *cmmcomp.exe*, sendo convertido para linguagem *Assembly*.
- (2) **Geração de Hardware:** O código *Assembly* é traduzido pelo *asmcomp.exe* para a descrição de *hardware* em Verilog.

(3) **Verificação Sintática e Estrutural:** O *software* Icarus Verilog (*iverilog*) é invocado em segundo plano para verificar a integridade dos arquivos Verilog gerados.

4.4 Simulação e Visualização RTL (GTKWave e PRISM)

Uma vez verificados os arquivos Verilog, a AURORA automatiza a simulação funcional ao invocar o motor vvp do Icarus Verilog. Para garantir a compatibilidade e a correta interpretação dos dados, a interface processa as saídas por meio dos conversores internos *comp2gtkw.exe* e *float2gtkw.exe*, consolidando as informações em um arquivo de despejo de variáveis (.vcd). Esse fluxo culmina na análise visual via GTKWave, conforme ilustrado na Figura 5, que apresenta as formas de onda geradas e permite a validação da integridade lógica, bem como a inspeção da cronologia dos sinais no circuito.

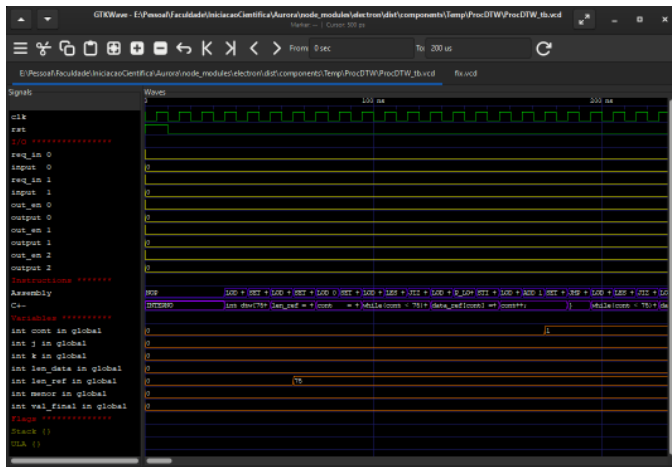


Figura 5. Formas de onda digitais exibidas no GTKWave após simulação comportamental do circuito.

Para a etapa de validação, a AURORA integra o PRISM (*Processor Rendering Interface for Schematic Models*), um visualizador RTL proprietário que compartilha a mesma base tecnológica (*Electron*). O PRISM utiliza o YOSYS (*Yosys Open SYNthesis Suite*) para extrair o *netlist* lógico a partir do código Verilog, atuando em conjunto com a ferramenta *netlistsvg* para renderizar a hierarquia de componentes e portas em diagramas vetorizados (SVG – *Scalable Vector Graphics*). A Figura 6 ilustra essa ferramenta em operação, exibindo a hierarquia de módulos de um circuito gerado pelo fluxo de compilação.

Cabe destacar que a síntese lógica final e o mapeamento físico do circuito *top-level* para o *chip* da FPGA extrapolam o escopo do ecossistema SAPHO, devendo ser realizados por meio das ferramentas de síntese proprietárias dos respectivos fabricantes.

5. COMPARAÇÃO ENTRE A AURORA E A IDE V1

A evolução do fluxo de projeto do ecossistema SAPHO pode ser observada ao se contrastar a AURORA com sua predecessora, a IDE V1. A Figura 7 apresenta a interface da V1, desenvolvida em C#, que se caracterizava por uma estrutura monolítica voltada à prova de conceito dos compiladores. Visualmente, a V1 consistia em janelas

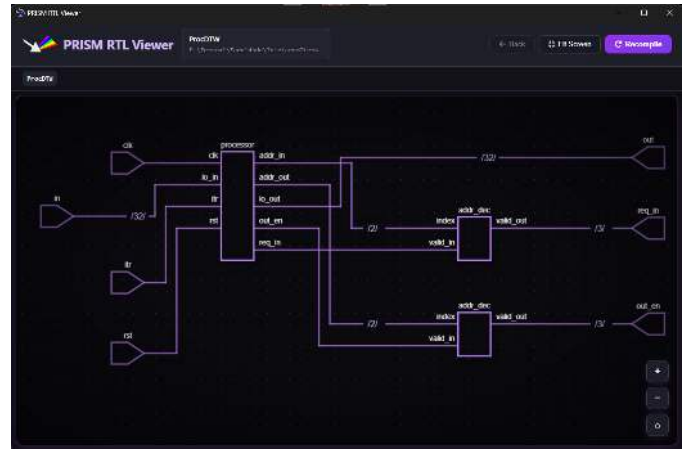


Figura 6. Visualizador RTL PRISM exibindo a hierarquia de componentes do circuito gerado.

estáticas, não responsivas e de fundo predominantemente branco, com um editor de texto básico e um único painel de console.

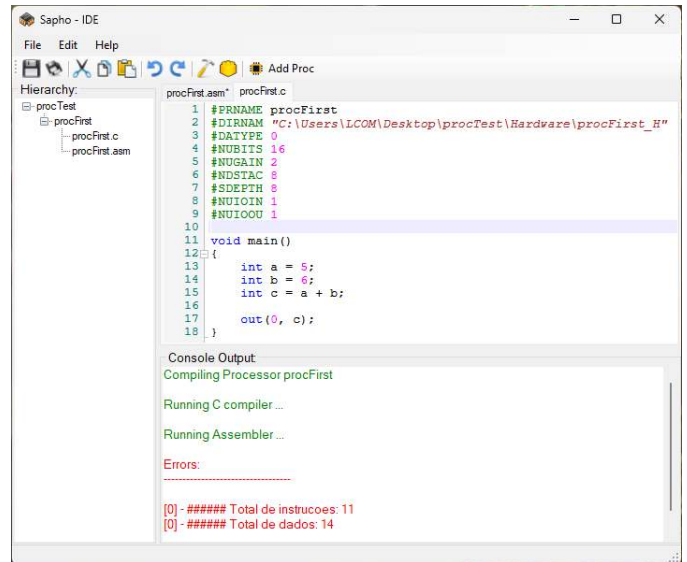


Figura 7. Interface da IDE V1, desenvolvida em C#, com editor de texto e painel de console único.

O escopo operacional da V1 era restrito: a ferramenta limitava-se a compilar o código de médio nível para gerar a descrição em *Assembly* e Verilog. Não havia integração com ferramentas de código aberto de terceiros, e a plataforma não oferecia verificação sintática de *hardware*, simulação de formas de onda ou visualização de RTL. Além disso, todo o registro de compilação (*log*) era agrupado de forma contínua em texto plano em uma única aba, dificultando o rastreamento de erros e limitando a escalabilidade do ambiente de pesquisa.

A IDE AURORA foi concebida para superar essas restrições por meio de uma arquitetura modular, responsiva e automatizada. Sua interface adota uma paleta de cores inspirada no fenômeno da *Aurora Borealis*, priorizando a ergonomia visual e a produtividade. O desenvolvimento de código é estruturado sobre o *Monaco Editor*, integrando ao ecossistema SAPHO recursos como destaque de sintaxe (*syntax highlighting*), autocompletar e navegação otimizada.

O principal avanço da AURORA em relação à V1 reside na automação do fluxo de compilação. Enquanto a versão anterior exigia a transição manual para ferramentas externas, a nova IDE permite que toda a compilação seja executada etapa a etapa – tradução do C $\pm$, geração do *Assembly*, e assim por diante – ou de forma integral por meio da função *Full Build*. Ao acionar esse comando, a interface coordena automaticamente todo o *pipeline*: desde a compilação do código-fonte (.cmm), passando pela verificação estrutural, até a simulação, que invoca e exibe os resultados no GTKWave.

Para gerenciar essa cadeia sem sobrecarregar o usuário, a AURORA substituiu o console único da V1 por um sistema de terminais dedicados e interativos: TCMM, TASM, TIVERI, TWAVE e TPRISM. Cada etapa do processo de síntese e simulação direciona seus resultados ao terminal correspondente. Essa separação é complementada por um esquema de cores semântico nos *logs* – vermelho para erros, laranja para advertências, verde para sucessos e azul para informações gerais –, permitindo identificar e tratar ocorrências de compilação com rapidez. Adicionalmente, os terminais fornecem informações sobre o consumo de recursos lógicos antes mesmo da prototipagem física na FPGA.

6. CONCLUSÃO

O desenvolvimento de algoritmos complexos em FPGAs frequentemente encontra dificuldades na integração de processadores embarcados e no desperdício de área lógica gerado por arquiteturas tradicionais. Para aprimorar o ciclo de projeto do processador dinâmico SAPHO, este trabalho apresentou a IDE AURORA, uma plataforma desenvolvida em *Electron* que unifica e moderniza o fluxo de desenvolvimento. Em comparação com a IDE V1, que apresentava escopo restrito e baixa automação, a AURORA atua como um ambiente responsivo, modular e integrado.

A plataforma simplifica o controle da cadeia de compiladores proprietária YANC de forma automatizada: por meio da função *Full Build*, o usuário avança da codificação em C $\pm$ até a extração do *netlist* RTL no PRISM e a visualização de formas de onda no GTKWave em uma única operação. O sistema de terminais dedicados e o esquema de cores semântico nos *logs* proporcionam rastreamento de erros eficiente e permitem estimar o consumo de recursos lógicos do *hardware* antes da etapa de roteamento físico.

Disponível como projeto de código aberto, a AURORA atende às demandas de pesquisa do laboratório NIPS-CERN, dando suporte a projetos de pós-graduação e sendo introduzida no currículo de graduação em Engenharia Elétrica da UFJF. Essa inserção indica que a ferramenta se adequa tanto ao desenvolvimento de pesquisas em arquiteturas customizáveis quanto ao ensino de projeto de *hardware* reconfigurável.

AGRADECIMENTOS

Os autores agradecem ao Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN) da Universidade Federal de Juiz de Fora (UFJF) pelo suporte institucional e pela infraestrutura que viabilizou o desenvolvimento deste trabalho. Agradecem também ao CERN pela parceria

que fundamenta as linhas de pesquisa do laboratório. Este trabalho contou com o apoio financeiro do Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq), da Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) e da Fundação de Amparo à Pesquisa do Estado de Minas Gerais (FAPEMIG).

REFERÊNCIAS

- Andrade Filho, L.M., Afonso, C.A.A., Kapisch, E.B., Castro, A.L.B., Rimes, S.M., and Libotte, G.B. (2025). Implementação de fractal em fpga usando o processador soft-core sapho. In *Encontro Nacional de Modelagem Computacional (ENMC 2025)*. URL https://www.nipscern.com/publications/articles/ENMC_2025_Fractal.pdf. Disponível em: https://www.nipscern.com/publications/articles/ENMC_2025_Fractal.pdf.
- ATLAS Collaboration (2008). The ATLAS experiment at the CERN large hadron collider. *Journal of Instrumentation*, 3(08), S08003. doi:10.1088/1748-0221/3/08/S08003. URL <https://doi.org/10.1088/1748-0221/3/08/S08003>.
- Compton, K. and Hauck, S. (2002). Reconfigurable computing: a survey of systems and software. *ACM Comput. Surv.*, 34(2), 171–210. doi:10.1145/508352.508353. URL <https://doi.org/10.1145/508352.508353>.
- Dirac, P.A.M. (1958). *The Principles of Quantum Mechanics*. Oxford University Press, Oxford, 4 edition.
- Donnelly, C. and Stallman, R. (2021). *Bison: The Yacc-compatible Parser Generator*. Free Software Foundation. URL <https://www.gnu.org/software/bison/manual/bison.pdf>. Version 3.8.1.
- Kapisch, E.B., Aguiar, M.S., and Filho, Luciano Manhães de Andrade Filho, L.R.M. (2026). Sapho—scalable-architecture processor for hardware optimization: An fpga customizable implementation approach. *IEEE Access*, 14, 6820–6832. doi:10.1109/ACCESS.2026.3652855.
- Levine, J. (2009). *flex & bison: Text Processing Tools*. O'Reilly Media.
- Núcleo de Instrumentação e Processamento de Sinais (NIPS) (2026). Sapho - scalable architecture processor for hardware optimization. <https://www.nipscern.com/projects/sapho>. Acessado em: 25 mar. 2026.
- Ribeiro, L.d.G. (2025). *Simulador de pulsos do TileCal em tempo real utilizando o processador SAPHO e FPGA*. Dissertação de mestrado, Universidade Federal de Juiz de Fora (UFJF), Juiz de Fora, MG, Brasil. URL <https://www2.ufjf.br/ppee/2025/08/25/defesa-de-dissertacao-de-mestrado-leandro-da-graca-ribeiro/>.
- Taivalsaari, A. and Mikkonen, T. (2017). The web as a software platform: Ten years later. In *Proceedings of the 13th International Conference on Web Information Systems and Technologies - Volume 1: WEBIST.*, 41–50. INSTICC, SciTePress. doi:10.5220/0006234800410050.
- Yang, Z., Chung, S.P., Chen, J., Zhang, R., Saltaformaggio, B., and Lee, W. (2025). Coindef: A comprehensive code injection defense for the electron framework. In *2025 IEEE Symposium on Security and Privacy (SP)*, 3127–3144. doi:10.1109/SP61157.2025.00195.